Iris Data Classification using a Support Vector Machine

Daniel Meister

10/16/2025

Loading the Packages

First make sure we have all the functionality from the `tidyverse` available:

```
library(tidyverse)
```

```
## -- Attaching packages --------------------------------------- tidyverse 1.3.0 --
```

```
## v ggplot2 3.3.5     v purrr   0.3.4
## v tibble  3.1.6     v dplyr   1.0.7
## v tidyr   1.1.0     v stringr 1.4.0
## v readr   1.3.1     v forcats 0.5.0
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
```

```
theme_set(theme_bw())
```

We are going to use the `e1071` package for this example and also some helper functions from `caret`

```
library(e1071)
library(caret)
```

```
## Loading required package: lattice
```

```
##
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
##
##     lift
```

Loading the Data

Load the `iris` data set into the environment

```
data(iris)
```

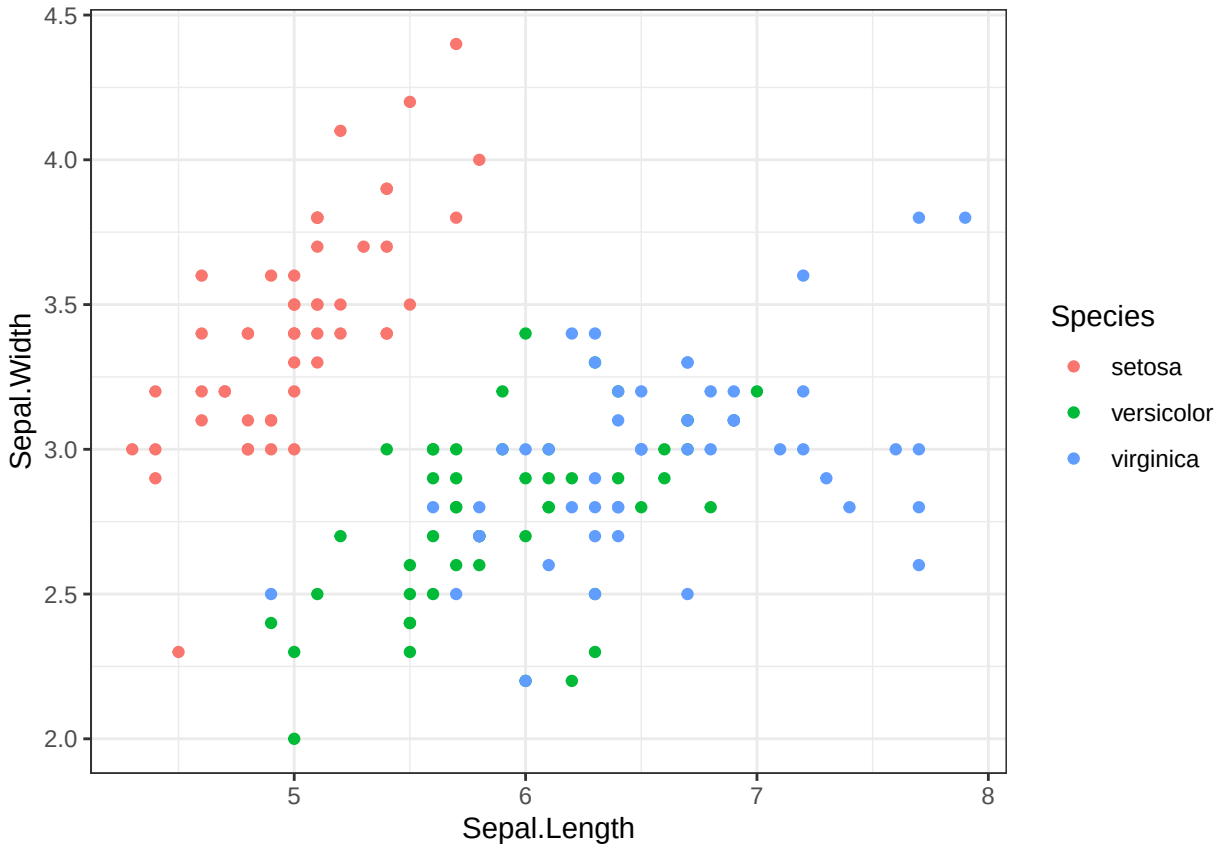
and let's have a look a the structure.

```
str(iris)
```

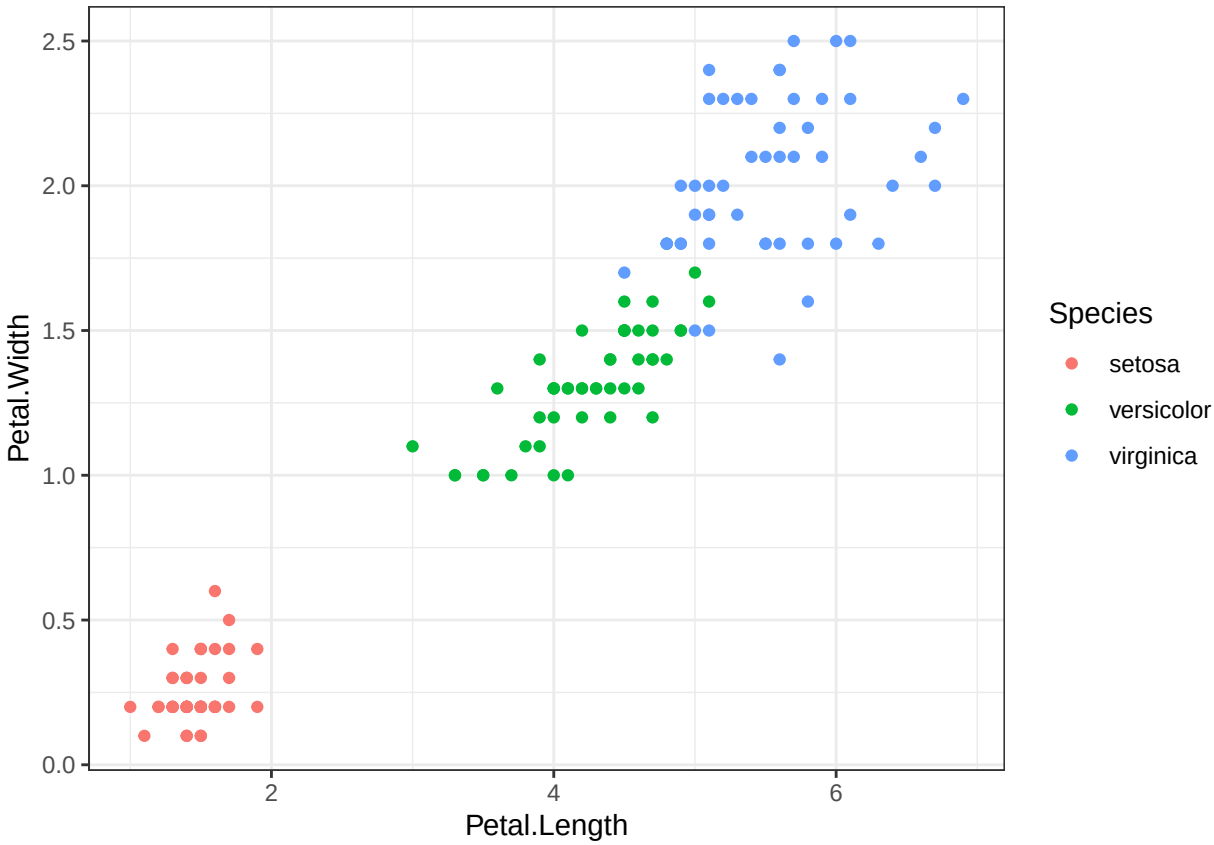
```
## 'data.frame':    150 obs. of  5 variables:
##  $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##  $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##  $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##  $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
##  $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

Have a quick look at the Data

```
iris %>%
  ggplot(aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_point()
```




```
iris %>%
  ggplot(aes(x = Petal.Length, y = Petal.Width, color = Species)) +
  geom_point()
```



Prepare the Data for Training

```
set.seed(123)
indices <- createDataPartition(iris$Species, p=.85, list = F)
```

Create some easy Variables to access Data

```
train <- iris %>%
  slice(indices)
test_in <- iris %>%
  slice(-indices) %>%
  select(-Species)
test_truth <- iris %>%
  slice(-indices) %>%
  pull(Species)
```

Train the Support Vector Machine

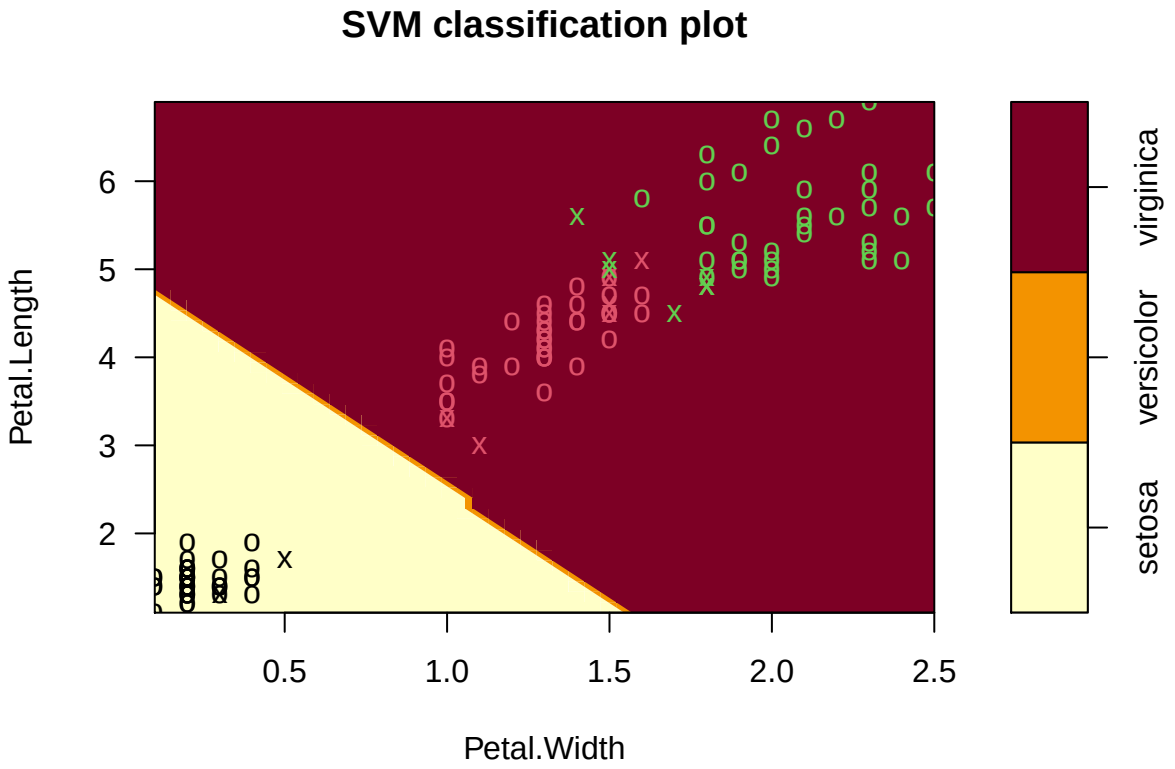
Call the `svm` function using the default `cost = 10` parameter.

```
set.seed(123)
iris_svm <- svm(Species ~ ., train, kernel = "linear", scale = TRUE, cost = 10)
```

Of course the visualization is a bit difficult as we have some classification regions in four dimensions.

So plot first the resulting classification for the two Petal leaf dimensions

```
plot(iris_svm, train, Petal.Length ~ Petal.Width)
```



And for the Sepal leaf dimensions (of course you need to slice the other dimensions at a reasonable point)

```
plot(iris_svm, train, Sepal.Length ~ Sepal.Width, slice = list(Petal.Length = 4.5, Petal.Width = 1.75))
```

Make Predictions

```
test_pred <- predict(iris_svm, test_in)
table(test_pred)
```

```
## test_pred
##     setosa versicolor  virginica
##          7          6          8
```

Evaluate the Results

```
conf_matrix <- confusionMatrix(test_pred, test_truth)
conf_matrix
```

```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   setosa versicolor virginica
##   setosa          7          0         0
##   versicolor      0          6         0
##   virginica       0          1         7
##
## Overall Statistics
##
##                Accuracy : 0.9524
##                  95% CI : (0.7618, 0.9988)
##     No Information Rate : 0.3333
##     P-Value [Acc > NIR] : 4.111e-09
##
##                   Kappa : 0.9286
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: setosa Class: versicolor Class: virginica
## Sensitivity                 1.0000            0.8571           1.0000
## Specificity                 1.0000            1.0000           0.9286
## Pos Pred Value              1.0000            1.0000           0.8750
## Neg Pred Value              1.0000            0.9333           1.0000
## Prevalence                  0.3333            0.3333           0.3333
## Detection Rate              0.3333            0.2857           0.3333
## Detection Prevalence        0.3333            0.2857           0.3810
## Balanced Accuracy           1.0000            0.9286           0.9643
```

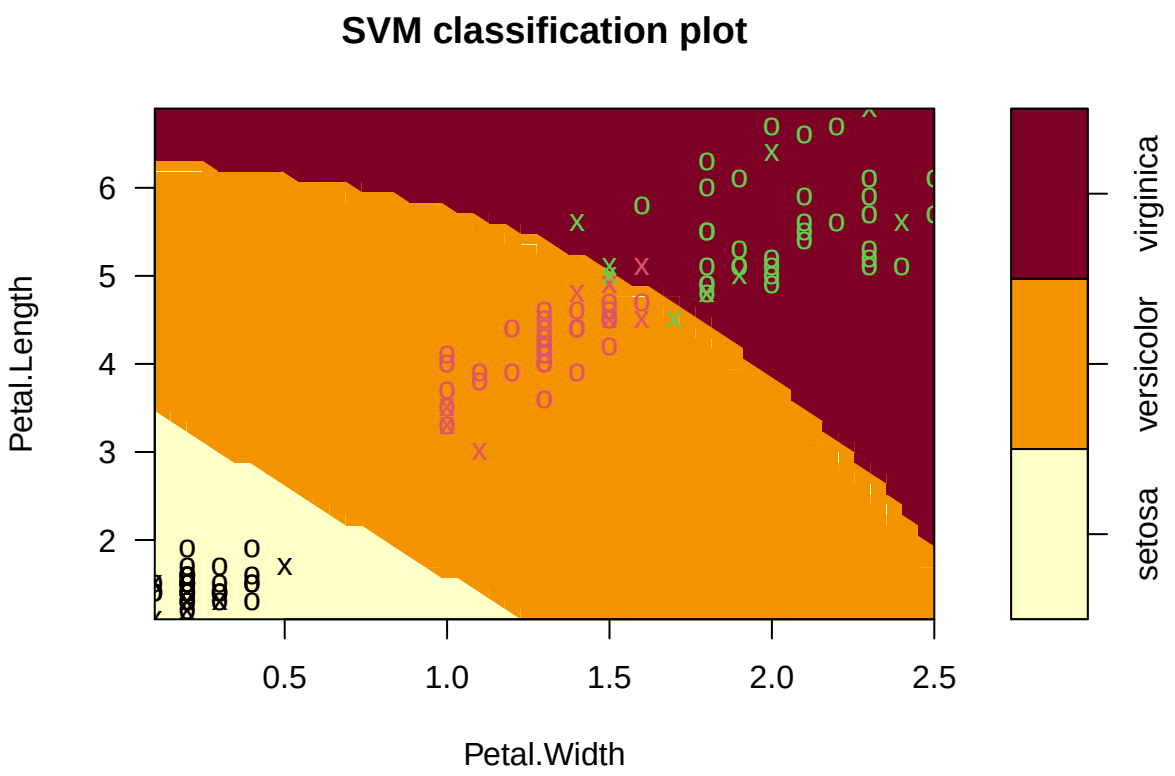
So we have a resulting SVM with a prediction accuracy of around 95% (of course we would also need to do some cross validation to ensure we were not simply "lucky" with the train/test split).

Try again with other parameters

```
set.seed(123)
iris_svm2 <- svm(Species ~ ., train, kernel = "radial", scale = TRUE, cost = 100)
summary(iris_svm2)
```

```
##
## Call:
## svm(formula = Species ~ ., data = train, kernel = "radial", cost = 100,
##     scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  100
##
## Number of Support Vectors:  25
##
##  ( 6 10 9 )
##
##
## Number of Classes:  3
##
## Levels:
##  setosa versicolor virginica
```

```
plot(iris_svm2, train, Petal.Length ~ Petal.Width, slice = list(Sepal.Length = 6, Sepal.Width = 3))
```




```
plot(iris_svm2, train, Sepal.Length ~ Sepal.Width, slice = list(Petal.Length = 4.5, Petal.Width = 1.75))
```


```
test_pred <- predict(iris_svm2, test_in)
table(test_pred)
```

```
## test_pred
##     setosa versicolor  virginica
##          7          6          8
```

```
conf_matrix <- confusionMatrix(test_pred, test_truth)
conf_matrix
```

```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   setosa versicolor virginica
##   setosa          7          0         0
##   versicolor      0          6         0
##   virginica       0          1         7
##
## Overall Statistics
##
##                Accuracy : 0.9524
##                  95% CI : (0.7618, 0.9988)
##     No Information Rate : 0.3333
##     P-Value [Acc > NIR] : 4.111e-09
##
##                   Kappa : 0.9286
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
```

```
##
##                      Class: setosa Class: versicolor Class: virginica
## Sensitivity                 1.0000            0.8571           1.0000
## Specificity                 1.0000            1.0000           0.9286
## Pos Pred Value              1.0000            1.0000           0.8750
## Neg Pred Value              1.0000            0.9333           1.0000
## Prevalence                  0.3333            0.3333           0.3333
## Detection Rate              0.3333            0.2857           0.3333
## Detection Prevalence        0.3333            0.2857           0.3810
## Balanced Accuracy           1.0000            0.9286           0.9643
```

So we have here the same accuracy but with much more complex boundaries (radial kernels and higher costs) so we might be overfitting here a bit more.